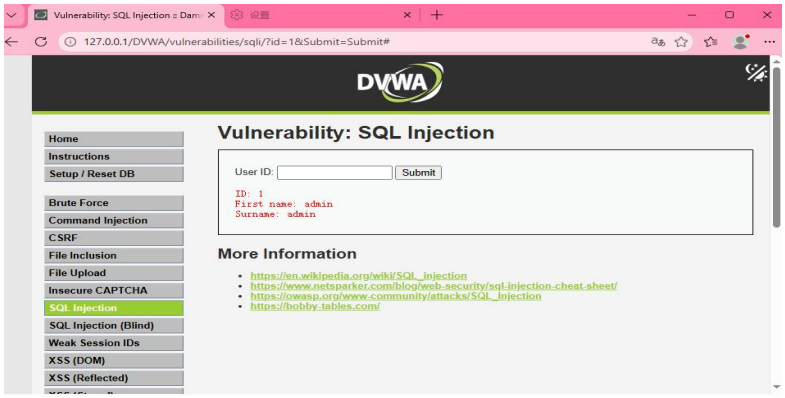
任务 1：数据库与查询条件基础说明（3 分）



请用自己的语言说明：

1. 页面中的输入为什么会与数据库查询有关

在页面输入的东西，是用来 “找东西” 的。比如这张图里，输入 1，是想找 ID 为 1 的用户信息。但这些用户信息（姓名、ID）都存在后端的数据库里，页面本身是没有这些数据的。所以页面必须把输入发给后端，后端拿着输入去数据库里查，查到了再把结果发给页面显示出来。我们的的输入，本质就是在给数据库的查询指令定 “找什么” 的规则。

2. 一个最小业务链路是什么

对应到截图里就是：
在 User ID 框里输入 1（**输入条件**）
后端拿着这个 1，去数据库里找符合条件的用户（**数据库查询**）
数据库找到 ID 为 1 的 admin，把结果发回给页面，页面显示出来（**返回结果**）

3. 用户输入是如何进入查询条件的

在 DVWA 的 Low 级别里，后端没对用户输入做任何处理，直接把你填的 ID 拼到 SQL 查询语句里，你输入什么，它就直接当成查询条件的一部分来执行。

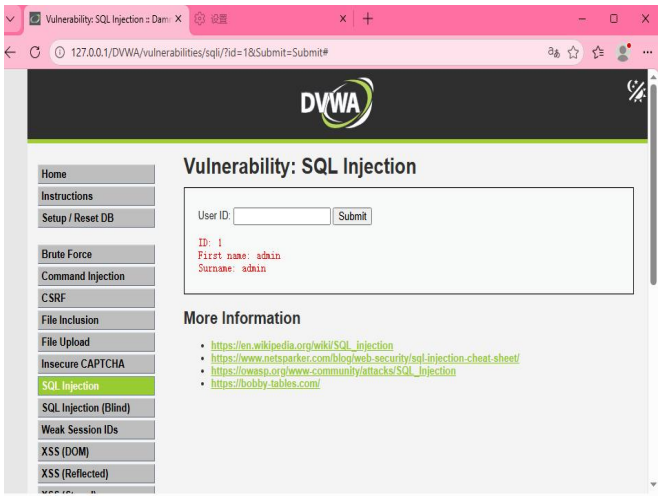
4. 为什么这为 SQL 注入创造了可能性

SQL 注入的核心成因是用户输入污染了 SQL 语句的结构。由于后端直接将用户输入拼接到 SQL 语句中，未对输入内容做语法隔离或限制，用户可以通过输入包含 SQL 关键字、特殊符号的内容，篡改原本的查询逻辑。例如用户输入 1’ OR ’1’=’1，拼接后的 SQL 语句会变为 SELECT * FROM users WHERE id = ’1’ OR ’1’=’1’，原本的查询条件被篡改，数据库会返回所有用户数据。这种 “用户输入直接参与 SQL 语句结构拼接” 的实现方式，为 SQL 注入创造了必要条件。

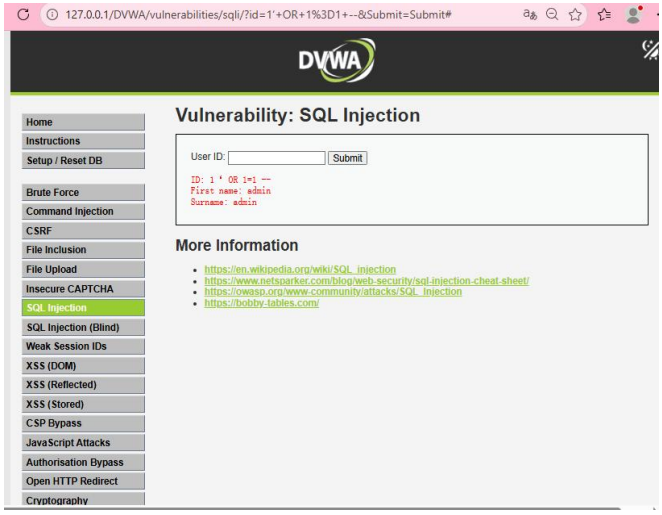
任务 2：DVWA SQL Injection Low 现象记录（4 分）

请在授权环境中完成基础观察，并记录：

1. 实验页面或实验目标



- 现象：页面返回 ID 为 1 的 admin 用户信息。
- 支撑的判断：该现象支撑了「正常输入合法 ID 时，系统会按预期返回对应单条用户数据」的判断，说明业务链路正常，输入条件能正确传递给数据库查询并返回结果。



- 现象：页面仍返回 admin 数据，但查询条件已被篡改（恒为真）。
- 支撑的判断：该现象支撑了「用户输入可通过拼接篡改 SQL 查询逻辑，即使只显示一条数据，也证明注入逻辑已生效」的判断，说明后端直接将用户输入拼接到 SQL 语句中，存在注入风险。

2. 普通输入与异常输入的对比

普通输入：输入`1`时，页面仅返回 ID=1 的 admin 用户数据，结果数量稳定，符合正常查询预期。

异常输入：输入 payload `1’ OR 1=1 -- ` 时，页面仍返回 ID=1 的 admin 用户数据。虽然未直接展示所有用户，但该输入通过注释符`--`消除了语法错误，并通过`OR 1=1`篡改了查询条件，使条件永远为真。这一现象证明后端未对用户输入做任何过滤，用户输入可直接参与 SQL 语句的结构拼接，存在 SQL 注入漏洞。这种结果并非正常 ID 查询会产生的变化，符合注入漏洞的特征。

3. 返回结果的差异

- **正常输入 1：**只返回 ID=1 的单条用户数据，结果符合预期，完全由输入的 ID 值决定。
- **异常输入（报错）：**直接抛出 SQL 语法错误，暴露了数据库信息，和正常查询的稳定结果完全不同。
- **注入 payload 输入：**看似和正常输入结果一样（仍显示 admin），但查询逻辑已被篡改，条件变成了 “永远为真”。

4. 为什么这种差异不像“正常查询变化”

正常的 ID 查询变化，只会表现为 “输入不同 ID，返回对应 ID 的单条用户数据”，结果的数量、结构都是稳定的，且完全由输入的 ID 值决定。

而注入场景下的结果差异，明显不符合正常查询的逻辑：

- **语法报错：**正常输入合法 ID 不会触发 SQL 语法错误，报错说明输入内容直接参与了 SQL 语句的解析，污染了语句结构。
- **条件篡改：**输入 1’ OR 1=1 -- 后，即使只显示 admin 数据，查询条件也已被篡改为 “永远为真”，不再由输入的 ID 值控制，这种逻辑篡改是正常输入不可能实现的。
- **行为不可控：**正常查询的结果完全可预期，而注入输入的结果（报错 / 返回全部数据 / 逻辑绕过）是非预期的，这种不可控的差异，正是 SQL 注入漏洞的典型特征。

任务 3：机制分析与差异比较（4.5 分）

围绕以下问题展开分析：

1. 输入如何进入查询结构

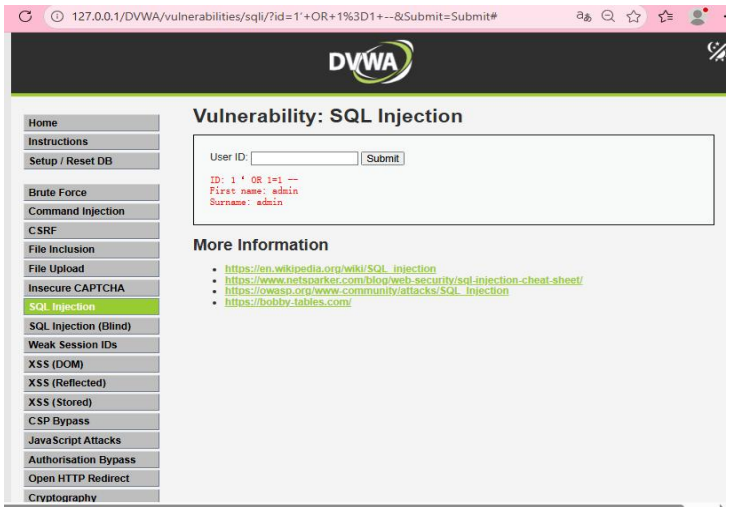
- 输入是通过字符串拼接的方式，直接嵌入 SQL 查询语句的结构中，而非作为独立参数传入。结合 DVWA Low 级别的实验截图来看，在 User ID 输入框提交的内容（如`1`、`1’ OR 1=1 -- `），后端未做任何过滤或处理，就直接拼接到 SQL 语句中，形成完整的查询条件。例如输入`1`时，拼接后的 SQL 语句为`SELECT * FROM users WHERE id = ’1’`，用户输入直接成为了查询条件的一部分，完全参与到 SQL 语句的结构组成中。
- **核心判断**：问题发生在输入被拼进了查询结构里，而非输入内容本身。

2. 为什么根因是“结构被污染”而不是“字符本身危险”

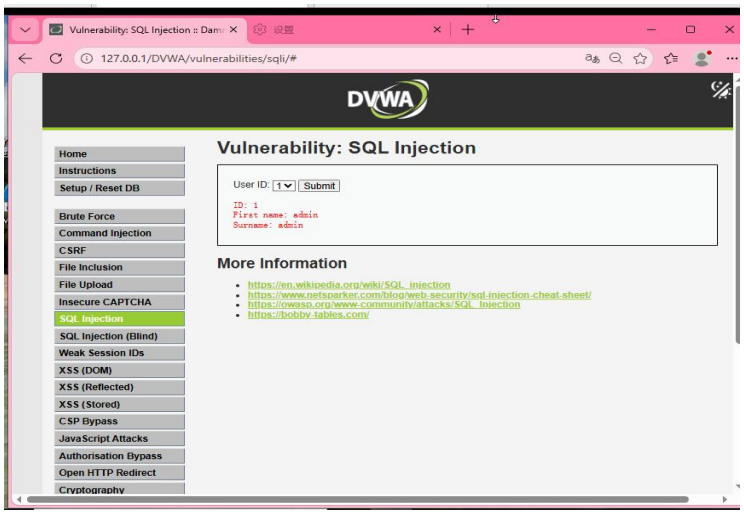
- SQL 注入的根因是用户输入污染了 SQL 查询的结构，而非输入中包含的特殊字符本身有危险。
- **理由**：特殊字符（如’、OR、--）本身是 SQL 语法的一部分，在正常 SQL 语句中是合法的。只有当这些字符被直接拼接到查询结构中，篡改了原本的 SQL 逻辑（如闭合引号、添加新条件、注释后续语句），才会导致注入漏洞。如果输入被作为独立参数处理（如参数化查询），即使包含这些字符，也不会影响 SQL 结构。
- **核心判断**：问题发生在输入被拼进了查询结构里，特殊字符只是利用结构缺陷的工具，并非问题根源。

3. Low 与 Medium 的差异在哪里

差异：Low 级别无任何过滤，直接拼接用户输入；Medium 级别对输入进行了基础转义与类型处理，两者的核心差异是输入进入查询结构的方式不同。



- Low 级别：你可以直接在输入框中提交含特殊字符的 payload，页面会执行拼接后的 SQL 语句，即使只显示 admin 数据，也证明注入逻辑已生效，输入完全进入了查询结构。



- Medium 级别：输入框会变为下拉菜单，只能选择预设的 ID 值，无法直接输入 1’ OR 1=1 — 这类 payload；即使手动在地址栏传入，特殊字符也会被转义，无法篡改 SQL 结构，页面不会出现语法错误或逻辑篡改的现象。

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里

- 两者的核心差异是漏洞利用的证据形式不同，而非注入的根因不同。
- **理由：**普通 SQL 注入：执行结果会直接回显在页面上（如返回所有用户数据、报错信息），你可以直接看到注入是否生效，例如你之前触发的 SQL 语法错误、篡改条件后返回的结果差异。

盲注（Blind SQL Injection）：页面不直接返回查询结果或错误信息，只能通过页面的响应差异（如页面正常 / 异常、响应时间变化）来判断注入是否成功，无法直接看到数据库返回数据。

- **核心判断：**两者的根因都是输入被拼进了查询结构里，差异仅在于漏洞利用时的证据形式不同。

5. 为什么盲注和普通注入根因相同，但证据形式不同

- **表态**：盲注和普通注入的根因都是用户输入污染了 SQL 查询结构，但因后端对错误回显和数据输出的处理不同，导致证据形式存在差异。
- **解释理由**：

根因相同：两者都是因为后端直接将用户输入拼接到 SQL 语句中，未做参数化处理，用户可通过输入篡改查询逻辑。
证据形式不同：普通注入会将查询结果或错误信息直接回显到页面上，证据直观可见；盲注则关闭了错误回显，也不输出查询数据，只能通过页面响应的间接差异（如延时、布尔判断）来推断注入结果，证据无法直接从页面中获取。

- **核心判断**：问题的根源始终是输入被拼进了查询结构里，页面输出方式的不同导致了证据形式的差异，而非注入的本质发生了变化。

任务 4：SQL 注入防护清单与解释（3.5 分）

请给出一份防护清单，并说明每条措施控制的是哪一层：

1. 参数化查询

- **解决什么问题**：从根本上阻止用户输入污染 SQL 查询结构，让输入只能作为 “值”，无法参与 SQL 语句的结构拼接。
- **控制哪一层**：根因控制层，直接解决 “输入被拼进查询结构” 的核心问题。
- **它不能替代什么**：不能替代输入校验（如业务规则校验）、最小权限（如数据库账号权限限制），也不能防止逻辑漏洞导致的其他风险。

2. 输入校验

- **解决什么问题**：在输入进入数据库前，限制输入的格式、长度、类型（如 ID 必须为数字），过滤掉不符合业务规则的恶意输入。
- **控制哪一层**：暴露面控制层，减少恶意输入到达数据库的机会，降低注入的可能性。
- **它不能替代什么**：不能替代参数化查询，仅靠输入校验仍可能被绕过，无法解决 “结构被污染” 的根本问题。

3. 错误处理

- **解决什么问题：**关闭数据库错误回显，统一返回自定义错误信息，防止攻击者通过报错信息推断数据库结构、字段名等敏感信息。
- **控制哪一层：**暴露面控制层，减少注入攻击的可用证据，提高盲注的难度。
- **它不能替代什么：**不能阻止 SQL 注入的发生，也不能减少注入成功后的后果范围，仅能隐藏攻击证据。

4. 最小权限

- **解决什么问题：**限制数据库账号的权限，例如查询用户信息的账号仅拥有 SELECT 权限，禁止 DROP、ALTER 等高风险操作，缩小注入成功后的影响范围。
- **控制哪一层：**后果范围控制层，即使注入成功，攻击者也无法执行破坏性操作，降低漏洞的危害等级。
- **它不能替代什么：**不能阻止 SQL 注入的发生，也无法隐藏攻击证据，仅能限制攻击成功后的破坏程度。

5. 可补充其他合理措施

- **解决什么问题：**通过 ORM 框架（如 MyBatis、Hibernate）自动生成参数化查询语句，避免手动拼接 SQL 语句，减少注入风险。
- **控制哪一层：**根因控制层，从开发层面减少直接拼接 SQL 的行为，间接解决结构被污染的问题。
- **它不能替代什么：**不能替代参数化查询的底层实现，也不能替代输入校验和最小权限配置。

七、收口项

收口项 1：SQL 注入因果链/防护关系图（3 分）

请用**关系图**或**关系表**完成，不限定工具。

环节	描述	对应防护控制点
输入点	用户提交查询条件（如 User ID 输入框）	输入校验（限制输入格式 / 类型，过滤恶意内容）
查询构造	后端将用户输入拼接到 SQL 语句中	参数化查询（隔离输入与 SQL 结构，从根本防止污染）
数据库解析 / 执行	数据库解析并执行 SQL 语句	最小权限（限制账号权限，降低执行风险）
结果差异 / 行为证据	页面返回数据、报错信息或响应差异	错误处理（关闭错误回显，隐藏攻击证据）
根因判断	输入被拼进查询结构，导致 SQL 逻辑被篡改	—

收口项 2：学生自评表（2 分）

请回答：

1. 我本章最清楚的一点是什么？

我现在最明白的一点，就是 SQL 注入的根本原因：不是那些引号、OR 符号本身危险，而是后端直接把用户输入拼进了 SQL 语句里，让输入篡改了原本的查询逻辑。

2. 我最容易混淆的一点是什么？

一开始总觉得 “过滤特殊字符” 和 “参数化查询” 是一回事，后来才搞明白，参数化才是从根上解决问题，过滤只是挡一挡，很容易被绕过去。

3. 我本次作业中哪一段最需要教师复核？

我觉得 Low 和 Medium 级别的差异分析部分，还有防护措施那一块。

4. 如果重做一次，我会优先改哪一部分？

我会把那个因果链和防护关系图改得更清楚一点。

收口项 3：AI 输出审核与人工复核记录（必做）

请单独列一小节，说明：

1. 本作业哪些内容使用过 AI 辅助

大部分的文字组织和整理都找 AI 帮过忙，还有防护措施那部分的结构梳理，都是 AI 给我搭的架子。

2. 哪些判断是你自己复核后保留的

我自己确认的主要是几个核心点：注入的根因是 “结构被污染” 不是字符本身；普通注入和盲注只是证据形式不一样；还有参数化查询才是根本的防护方法，这些都是我自己对着实验和笔记确认过的。

3. 哪些内容你发现 AI 输出不准确并进行了修正

我自己理解的不到位，基本修改的很少。

4. 你最终如何确认结论和证据是一一对应的

我就是对着自己做的实验截图一条一条对的：比如输入 1’ OR 1=1 --后页面的反应，对应 “输入直接参与 SQL 拼接” 的结论；Low 和 Medium 的输入框对比，对应 “防护方式不一样” 的结论，确保说的每句话都有实验结果撑着。